TECNOLOGÍAS PARA LA AUTOMATIZACIÓN · 4K3 · 2026

Bitácora del TPI de IoT
Grupo 24

Todo lo que hicimos para dejar el trabajo práctico funcionando: qué es cada parte, cómo se instaló, cómo se probó en una placa virtual y en la placa real, qué problemas aparecieron y cómo se resolvieron, y qué mediciones obtuvimos.

Contenido

1. Resumen en una página
2. Conceptos básicos, explicados simple
3. La consigna y cómo la cumple el código
4. La placa de la cátedra
5. Cómo funciona el Sketch 1 (envío)
6. Cómo funciona el Sketch 2 (lectura)
7. Bitácora: lo que hicimos, en orden
8. Todas las mediciones
9. Problemas y soluciones (resumen)
10. Qué hacer el día de la evaluación
11. Preguntas probables
12. Pendientes y advertencias
13. Anexo: archivos del proyecto y comandos usados

Integrantes	Zengaro, Tomas (403344) · Visintin, Juan (95843) · Bongiorno, Valentino (98495) · Peralta, Santino (403430) · Carrizo, Martin (400562) · Maurino, Gonzalo (401062)
Canal ThingSpeak	Channel ID 3546746 · thingspeak.mathworks.com/channels/3546746
Fecha de la bitácora	22 de octubre de 2026
Entrega	Miércoles 28/10, hasta las 23:59, en la UV

1. Resumen en una página

Qué es el TP. Son dos programas (en Arduino se llaman *sketches*) para una misma placa ESP32:

- **Sketch 1 – envío:** se carga por cable USB. Lee una perilla, dos pines táctiles y un sensor de temperatura y humedad; maneja dos LEDs; muestra todo en una pantallita; y cada 16 segundos manda 6 datos a **ThingSpeak** (una página web que guarda y grafica datos). Además deja la placa preparada para recibir el segundo programa **por WiFi** (eso se llama OTA).
- **Sketch 2 – lectura:** se carga por WiFi, reemplaza al primero, lee de ThingSpeak los datos que mandó el Sketch 1, los muestra en pantalla y cuenta cuántas veces se aprieta un pulsador.

> **Estado final (22/10/2026, en la placa real de la cátedra)**
> - ✓ Sketch 1 subido por USB y funcionando: perilla, touch, sensor, pantalla y envíos a ThingSpeak.
> - ✓ Sketch 2 subido **por WiFi (OTA)** y funcionando: leyó exactamente lo guardado en el canal y contó 31 pulsaciones.
> - ✓ Los dos programas compilan sin errores con la versión final que se entrega.

Los 5 momentos del trabajo

Momento	Qué pasó
1. Preparar la compu	Instalamos Arduino IDE 2, el soporte para ESP32 y 5 librerías. Los dos programas compilaron.
2. Crear ThingSpeak	Canal "TPI Grupo 24" con 6 campos, público. Probamos escribir y leer desde la compu: funcionó.
3. Simular sin placa	Con el simulador Wokwi corrimos los dos programas en una ESP32 virtual: envío, lectura y pulsador funcionaron. Touch y OTA no se pueden simular.
4. Probar en la placa real	Instalamos el driver USB, subimos el Sketch 1, **corregimos el touch** (no detectaba el dedo), y subimos el Sketch 2 por WiFi.
5. Dejar lista la entrega	WiFi de la facu (CIDS) en el código, integrantes en el encabezado, guion y presentación corregidos.

Qué se entrega (solo esto)

1. `sketch_envio_grupo_24.ino`
2. `sketch_lectura_grupo_24.ino`
3. `integrantes_grupo_24.txt` (nombres, legajos, Channel ID y Write API Key)

No hace falta ningún PDF ni informe: la consigna pide sólo esos tres archivos. Esta bitácora es para estudiar.

2. Conceptos básicos, explicados simple

Concepto	Qué es
ESP32	Una plaquita con un microcontrolador (una "computadora muy chiquita") que además tiene WiFi y Bluetooth. Es el cerebro del TP.
Sketch	Así se llama un programa de Arduino. Es un archivo `.ino` que tiene que estar dentro de una carpeta con el mismo nombre.
Compilar	Traducir el código que escribimos a un idioma que entiende la ESP32. Si hay errores de escritura, aparecen acá. En el IDE es el botón ✓ (Verificar).
Subir (upload)	Pasar el programa compilado a la placa. En el IDE es el botón → (Subir).
Puerto COM	Cuando enchufás la placa por USB, Windows le asigna un "puerto serie" con nombre COM + número (a nosotros nos tocó **COM11**). Por ahí se sube el programa y se leen los mensajes.
Driver	Un programita de Windows que le enseña a hablar con un dispositivo. La ESP32 usa un chip USB llamado **CP2102**, que necesita su driver.
Monitor serie	Una ventana donde se ven los mensajes que la placa escribe con `Serial.print(...)`. Se abre a **115200** baudios (velocidad).
setup() y loop()	Todo sketch tiene dos partes: `setup()` se ejecuta **una vez** al arrancar (preparar cosas); `loop()` se repite **para siempre**.
millis() vs delay()	`delay(1000)` frena todo 1 segundo. `millis()` dice cuántos milisegundos pasaron desde que arrancó, y sirve para hacer cosas "cada tanto" sin frenar. Usamos `millis()` para que el loop quede libre y pueda atender el OTA.
WiFi 2.4 GHz	La ESP32 sólo se conecta a redes de 2.4 GHz, no de 5 GHz. Una misma red (como CIDS) puede tener las dos bandas.
ThingSpeak	Página web gratuita (de MathWorks) que guarda datos en "canales" y los grafica. Cada canal tiene hasta 8 "campos" (fields).
Channel ID / Write API Key	El número que identifica el canal, y la "clave para escribir" en él. Para leer un canal **público** no hace falta clave.
OTA (Over The Air)	Subir un programa a la placa **por WiFi** en vez de por cable. El Sketch 1 abre un "puerto de red" llamado `esp32-grupo24` que aparece en el IDE.
PWM	Forma de regular el brillo de un LED prendiéndolo y apagándolo muy rápido. Cuanto más tiempo prendido, más brillo.
12 bits	Que los valores van de 0 a 4095 (2^{12} = 4096 niveles). La consigna pide trabajar con 12 bits.
ADC	Conversor analógico-digital: mide un voltaje y lo convierte en número. Con el potenciómetro, da 0 a 4095.

Touch capacitivo	Algunos pines de la ESP32 detectan el dedo: al tocarlos, el número que devuelve `touchRead()` **baja**.
Interrupción	Un aviso inmediato del hardware: "¡apretaron el botón!". Así no se pierden pulsaciones aunque el programa esté ocupado.
Antirrebote	Al apretar un botón, el contacto "vibra" y genera varios pulsos. Se ignoran los que llegan muy juntos (menos de 200 ms).
arduino-cli	La versión "de consola" del Arduino IDE. Hace lo mismo que los botones (compilar, subir, instalar librerías) pero con comandos. La usamos para automatizar todo.
Wokwi	Simulador online de placas: corre el mismo programa en una ESP32 virtual con componentes virtuales.

3. La consigna y cómo la cumple el código

Fuente: "Trabajo Practico de IoT 4K3 – Comunicación Bidireccional ESP32–ThingSpeak con Programación OTA" (archivo `TPI 4K3.pdf`).

Sketch 1 (se carga por USB)

Pide la consigna	Cómo lo hace el código	Probado
Cargarse por USB y habilitar OTA	`ArduinoOTA` con nombre `esp32-grupo24`	**Placa real**
Potenciómetro → brillo del LED integrado, PWM en pantalla, 12 bits	ADC de 12 bits (0–4095) va directo al PWM de 12 bits	**Placa real + simulación**
Temperatura y humedad en pantalla en tiempo real	DHT22 leído cada 2 s; pantalla redibujada cada 200 ms	**Placa real + simulación**
LED externo con dos pines touch (uno sube, otro baja), % en pantalla, 12 bits	D13 sube y D4 baja, 1 % cada 50 ms; PWM 0–4095	**Placa real** (después de corregir)
Cada 16 s enviar: aleatorio 100–500, temperatura, humedad, segundos desde el inicio, PWM LED integrado, % LED externo	Los 6 campos en **una sola** escritura (`writeFields`)	**Placa real + simulación**
Canal público	Configurado en ThingSpeak (pestaña Sharing)	**Verificado**

Sketch 2 (se carga por OTA)

Pide la consigna	Cómo lo hace el código	Probado
Subirse desde el puerto OTA del Sketch 1	Mismo nombre de red; también deja OTA activo	**Placa real**
Leer datos del mismo canal	`readMultipleFields` cada 16 s, sin clave (canal público)	**Placa real + simulación**
Contar las pulsaciones del pulsador	Interrupción en GPIO19 + antirrebote de 200 ms	**Placa real (31) + simulación (3)**
Mostrar valores y pulsaciones de forma legible (puede ser 3 campos por pantalla rotativa)	2 pantallas (F1–F3 y F4–F6) que alternan cada 4 s	**Placa real + simulación**

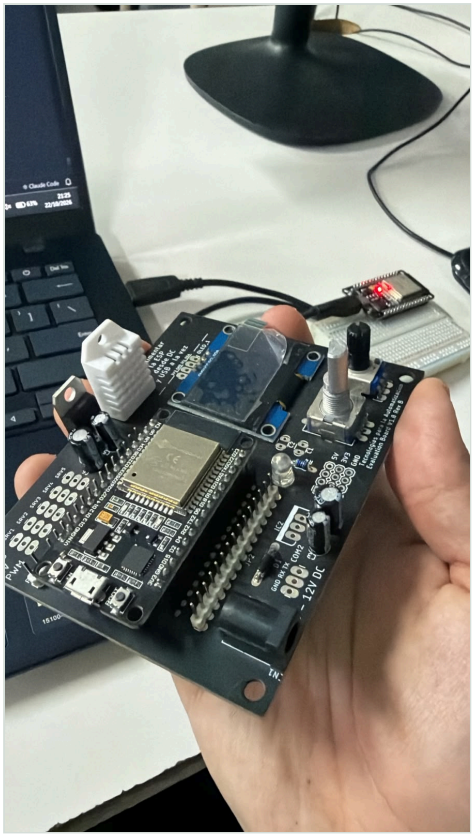
Lo que dice la consigna sobre la entrega y la evaluación

- Subir a la UV los dos `.ino` y un `.txt` con Apellido, Nombre y Legajo de los integrantes, más Channel ID y Write API Key.
- Fecha límite: **28 de octubre**.
- Fechas de evaluación posibles: **jueves 29/10, jueves 5/11, jueves 12/11, martes 17/11, jueves 19/11**. Se informa por Autogestión qué día le toca a cada grupo. **Tienen que estar los 6.**

- **Se evalúa únicamente la versión subida a la UV.** Para la Aprobación Directa, el código tiene que estar operativo en esta primera entrega (se aceptan errores menores).

> **Por qué importa "se evalúa sólo lo subido"**
> El nombre y la clave del WiFi están escritos dentro del código. Si el día de la evaluación no hay esa red, la placa no se conecta. Por eso decidimos dejar en el código la red de la facultad (**CIDS**), que es donde se evalúa.

4. La placa de la cátedra

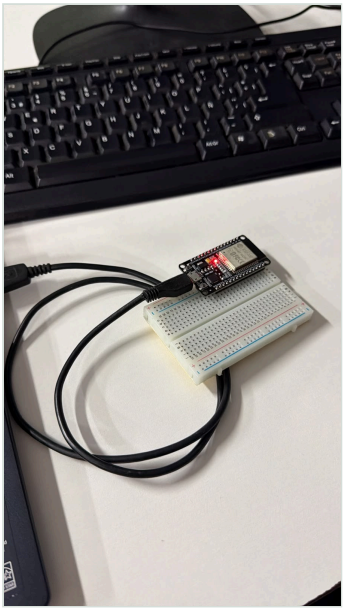


La placa de la cátedra ("Tecnologías para la Automatización – Evaluation Board V1.0 Rev B"). Al fondo, la ESP32 suelta en la protoboard, que **no** es la que se usa para el TP.

Componente	Pin de la ESP32	Dónde está / cómo se reconoce
Potenciómetro	GPIO32	La perilla **negra chiquita**
Sensor DHT22 (temperatura y humedad)	GPIO33	La cajita **blanca** con rejillas
LED integrado (azul)	GPIO2	Viene en la propia ESP32
LED externo	GPIO23	El LED redondo de la placa, cerca del centro
Touch "sube"	GPIO13 (T4)	Patita **D13**: fila del lado del sensor, 3ª desde el USB (VIN, GND, **D13**)
Touch "baja"	GPIO4 (T0)	Patita **D4**: otra fila, 5ª desde el USB (3V3, GND, D15, D2, **D4**)
Pulsador	GPIO19	El **eje plateado grande** (encoder): se aprieta hacia abajo, hace "click"
Pantalla OLED SH1106 128×64	SDA 21 / SCL 22, dirección 0x3C	La pantallita (sacarle el plástico protector)
Botones BOOT y EN	—	A cada lado del conector micro USB de la ESP32

> **Cuidado**
>
> - No enchufar nada en el conector negro de **12V DC** mientras está el USB: la placa indica no alimentar por los dos a la vez.
> - El relé y los pines SRV no se usan en este TP.

La ESP32 suelta vs. la placa de la cátedra



Primero tuvimos sólo una ESP32 suelta en una protoboard: sin perilla, sensor, pantalla, LED externo ni pulsador. Sirve para probar subida, WiFi, touch y OTA, pero no el TP completo.

En esa ESP32 suelta se veía el **LED azul prendido**: no era un error. El programa leía la "perilla" en GPIO32, pero como no había ninguna conectada, el pin quedaba "al aire" y daba números al azar, que se convertían en brillo.

5. Cómo funciona el Sketch 1 (envío)

Archivo: `sketch_envio_grupo_24/sketch_envio_grupo_24.ino`

Arranque (setup, una sola vez)

1. Prende la pantalla y configura el PWM de los dos LEDs en 12 bits.
2. **Calibra los pines touch**: lee 20 veces cada uno sin tocar y guarda el 90 % del promedio como "umbral". **Por eso no hay que tocarlos los primeros segundos.**
3. Se conecta al WiFi (muestra "Conectando WiFi..." en pantalla).
4. Activa el OTA con el nombre `esp32-grupo24`.
5. Prepara la conexión con ThingSpeak.

Ciclo infinito (loop)

Cada cuánto	Qué hace
Siempre, primero	Atiende el OTA (por si llega el Sketch 2)
Siempre	Lee la perilla y ajusta el LED azul
Cada 50 ms	Lee los touch: si D13 está tocado sube 1 %, si D4 está tocado baja 1 %
Cada 2 s	Lee temperatura y humedad (si falla, conserva el valor anterior)
Cada 200 ms	Redibuja la pantalla
Cada 16 s	Manda los 6 campos a ThingSpeak e imprime el resultado

Partes clave del código

Perilla → LED (los dos son de 12 bits, así que el valor pasa directo)

```
void leerPotenciometro() {
  pwmLedInt = analogRead(PIN_POTE);   // ADC de 12 bits: 0..4095, igual que el PWM
  pwmEscribir(PIN_LED_INT, CANAL_PWM_INT, pwmLedInt);
}
```

Envío: los 6 campos juntos en una sola escritura

```
ThingSpeak.setField(1, aleatorio);      // random(100, 501) -> 100..500
ThingSpeak.setField(2, temperatura);
ThingSpeak.setField(3, humedad);
ThingSpeak.setField(4, segundos);       // millis() / 1000
ThingSpeak.setField(5, pwmLedInt);
ThingSpeak.setField(6, porcentajeExt);
ultimoEstadoEnvio = ThingSpeak.writeFields(CANAL_ID, WRITE_API_KEY);
```

¿Por qué juntos? La cuenta gratuita de ThingSpeak acepta **una escritura cada 15 segundos**. Seis escrituras separadas serían rechazadas casi todas. Y por eso también el intervalo es de 16 s: 1 segundo de margen.

Lo que imprime por el monitor serie

```
Umbrales touch: +1017  -846
Conectando a WiFi....
Conectado. IP: 172.16.18.63
OTA listo: puerto de red 'esp32-grupo24'
Envio -> rnd=130 T=25.5 H=55.1 t=18s PWM=2833 ext=0% | OK
```

Lo que muestra la pantalla

```
TPI G24 - ENVIO
PWM LED int: 2833
LED ext: 0 %
Temp: 25.5 C
Hum:  55.1 %
Envio en 9s [OK]
IP 172.16.18.63
```

6. Cómo funciona el Sketch 2 (lectura)

Archivo: `sketch_lectura_grupo_24/sketch_lectura_grupo_24.ino`

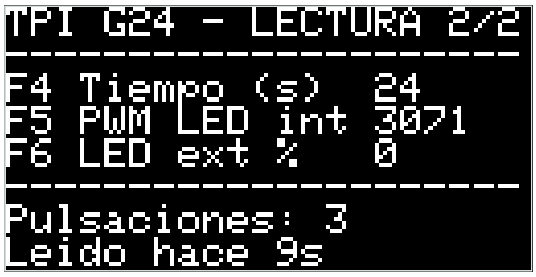
- **Arranque:** configura el pulsador (GPIO19) con resistencia *pull-up* (suelto = alto, apretado = bajo) y le engancha una **interrupción**; se conecta al WiFi, activa OTA y hace una primera lectura del canal.
- **Cada 16 s:** lee el último registro del canal con `readMultipleFields` y guarda los 6 valores.
- **Cada 4 s:** cambia entre la pantalla 1/2 (F1–F3) y la 2/2 (F4–F6).
- **Siempre abajo:** "Pulsaciones: N" y "Leido hace Xs".

El pulsador por interrupción, con antirrebote

```
void IRAM_ATTR alPresionar() {
  unsigned long ahora = millis();
  if (ahora - tUltimaPulsacion > ANTIRREBOTE_MS) {   // 200 ms
    pulsaciones = pulsaciones + 1;
    tUltimaPulsacion = ahora;
  }
}
...
pinMode(PIN_PULSADOR, INPUT_PULLUP);
attachInterrupt(digitalPinToInterrupt(PIN_PULSADOR), alPresionar, FALLING);
```

¿Por qué interrupción? Leer ThingSpeak tarda alrededor de un segundo. Si el botón se revisara en el loop, las pulsaciones de ese momento se perderían.

Lo que muestra la pantalla (captura de la simulación)



Pantalla 2/2 del Sketch 2 en Wokwi: campos F4–F6, "Pulsaciones: 3" y "Leido hace 9s".

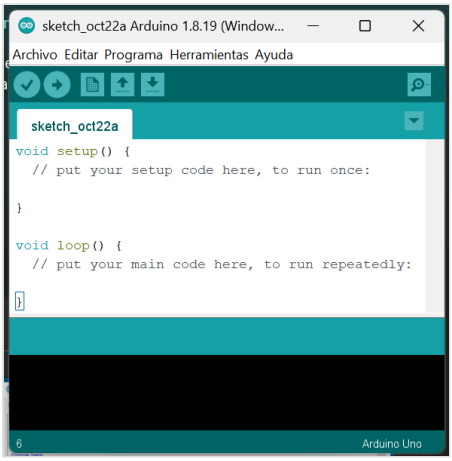
7. Bitácora: lo que hicimos, en orden

Etapa 1 · Entender el proyecto

El proyecto ya tenía los dos sketches, el `.txt` de integrantes, un instructivo ("PASO A PASO") y una guía (presentación HTML, guion y video). Faltaba completar el WiFi, el Channel ID y la Write API Key, que estaban como `COMPLETAR` / `XXXX`.

Etapa 2 · Instalar las herramientas

1. **Arduino IDE.** Lo instalamos desde la consola con `winget` (el instalador de programas de Windows): versión **2.3.10**. Había además una versión vieja (1.8.19) instalada desde la Microsoft Store, que conviene no usar.



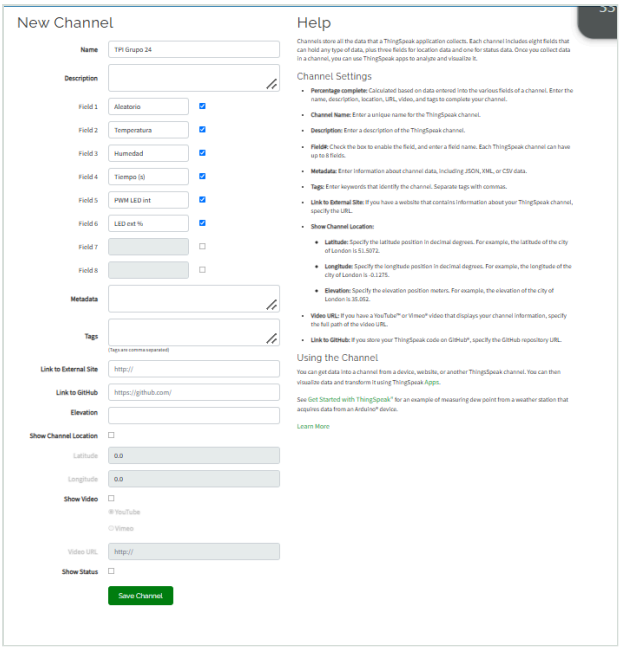
La versión vieja (Arduino 1.8.19, de la Microsoft Store). La que usamos es la 2.3.10.

2. **Soporte para ESP32 y librerías.** Con `arduino-cli` (viene dentro del IDE 2) agregamos la dirección de placas de Espressif e instalamos el paquete **esp32 3.3.12** (~2 GB). Después, las librerías: DHT sensor library 1.4.7, Adafruit Unified Sensor 1.1.15, Adafruit SH110X 2.1.15, Adafruit GFX 1.13.0, ThingSpeak 2.1.1 (y Adafruit BusIO 1.17.4, que se instaló sola como dependencia). Se usó la misma configuración del IDE, así que todo aparece también al abrirlo.

3. **Primera compilación** para la placa "DOIT ESP32 DEVKIT V1": los dos sketches compilaron sin errores. Ocupan 78 % (envío) y 76 % (lectura) de la memoria de programa.

Etapa 3 · Crear el canal de ThingSpeak

Creamos la cuenta con el mail institucional y el canal "TPI Grupo 24" con 6 campos, en el mismo orden que pide la consigna. Lo hicimos público (Sharing → "Share channel view with everyone").



Formulario "New Channel" con los 6 campos: Aleatorio, Temperatura, Humedad, Tiempo (s), PWM LED int, LED ext %.

Prueba desde la compu (sin placa), imitando lo que harían los sketches:

```
ESCRITURA (como Sketch 1)  -> respuesta "1"  (se guardó el registro n.º 1)
LECTURA sin clave (como Sketch 2):
{"entry_id":1,"field1":"321","field2":"24.5","field3":"55.0",
 "field4":"16","field5":"2048","field6":"50"}
```

Conclusión: Channel ID y Write API Key correctos, canal público, campos bien nombrados.

Etapa 4 · Simular la placa con Wokwi

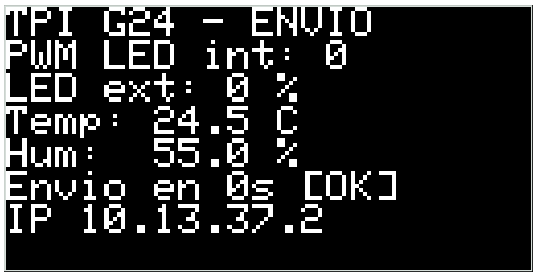
Como todavía no teníamos la ESP32, armamos una **placa virtual** en la carpeta `simulacion_wokwi/` : una ESP32 con pantalla, perilla, sensor y dos LEDs (envío) y otra con pantalla y botón (lectura), en los mismos pines que la real.

El script `preparar_simulacion.ps1` **copia** los sketches (sin tocar los originales) y les cambia sólo dos cosas: el WiFi pasa a `Wokwi-GUEST` (la red virtual del simulador) y la pantalla SH1106 pasa a SSD1306 (Wokwi no tiene la SH1106; son casi iguales). Después los compila.

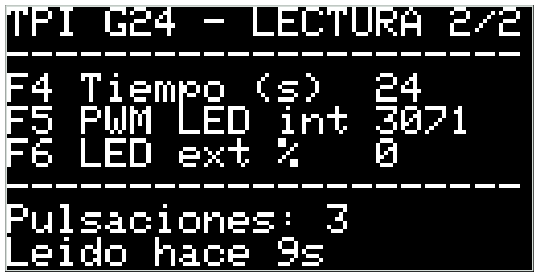
Con la herramienta de consola de Wokwi (`wokwi-cli` 0.29.0) corrimos las simulaciones y además "escenarios" automáticos que mueven la perilla, cambian el clima del sensor y aprietan el botón.

Simulación	Resultado
Envío, sin tocar nada	`Envio -> rnd=472 T=24.5 H=55.0 t=24s PWM=0 ext=0% | OK` y otro OK a los 40 s
Envío, perilla a 3/4 y clima 30.2 °C / 68 %	`Envio -> rnd=402 T=30.2 H=68.0 t=24s PWM=3071 ext=0% | OK` 3071 ≈ 3/4 de 4095 ✓
Lectura	`Lectura OK: 402 | 30.2 | 68.0 | 24 | 3071 | 0` — exactamente lo que mandó el envío ✓

Lectura, 3 clics en el botón	Pantalla: "Pulsaciones: 3" ✓



Pantalla del Sketch 1 en la simulación: valores, "[OK]" e IP virtual.



Pantalla del Sketch 2 en la simulación, después de 3 pulsaciones.

> **Límites de la simulación**
>
> - **Touch:** Wokwi no simula los pines táctiles (aparecían mensajes de error `touch: ... timeout` y el LED externo quedaba en 0 %).
> - **OTA:** subir por WiFi necesita la placa física y la compu en la misma red.
> - **Incidente:** una simulación se "colgó" porque le pedimos una foto de la pantalla para un momento posterior al final del escenario; la cuenta gratuita la cortó a los 5 minutos. Se resolvió alargando el escenario.

Etapa 5 · Revisar la consigna

Comparamos el código punto por punto con `TPI 4K3.pdf`: **cumple todo**. Aparecieron tres detalles de entrega:

1. La evaluación puede ser cualquiera de 5 fechas (no necesariamente el 19/11, como decía el guion).
2. Como se evalúa sólo lo subido, el WiFi del código tiene que ser el del lugar de evaluación.
3. Conviene poner los integrantes también en el encabezado de los `.ino`.

Etapa 6 · En la facu: conectar la placa

Problema 1: la placa aparecía con error y sin puerto COM

En el Administrador de dispositivos la placa figuraba como **"CP2102 USB to UART Bridge Controller"** con error código 28 = "no hay driver instalado".

- La notebook tiene procesador **ARM (Snapdragon X)**: no sirve cualquier driver, tiene que ser compatible con ARM64.
- La página de Silicon Labs (fabricante del chip) **bloqueaba la descarga** desde la consola ("Access Denied").
- Windows Update no ofreció ningún driver.
- **Solución:** lo bajamos del **Catálogo de Microsoft Update**: "Silicon Laboratories Inc. Ports Driver Update" versión **11.5.0.417**, firmado por Microsoft y con soporte ARM64. Lo instalamos con `pnputil` (con VS Code abierto como administrador).
- **Resultado:** "Silicon Labs CP210x USB to UART Bridge (**COM11**)" OK.

Problema 2: ¿la red de la facu es de 2.4 GHz?

La notebook estaba conectada a **CIDS** en 5 GHz. Revisando las redes cercanas vimos que CIDS también emite en **2.4 GHz** (canales 1 y 6), así que la ESP32 se puede conectar. Cargamos CIDS en los dos sketches.

Problema 3: "Wrong boot mode detected (0x13)"

Al subir el Sketch 1 la placa no entraba en **modo carga** (se quedaba en `Connecting.....`). Es muy común en estas ESP32. **Solución:** mantener **BOOT**, apretar y soltar **EN**, soltar **BOOT**. Después de eso, la carga funcionó (y en las siguientes ya no hizo falta).

Problema 4: la carpeta se movió

En el medio, el proyecto se movió a la subcarpeta `TP-de-TPA`. Los comandos fallaban con "no such file or directory" hasta que actualizamos la ruta.

Etapa 7 · Sketch 1 en la placa real

Subido por USB. Primeros envíos:

```
Conectado. IP: 172.16.18.63
OTA listo: puerto de red 'esp32-grupo24'
Envio -> rnd=130 T=25.5 H=55.1 t=18s PWM=2833 ext=0% | OK
Envio -> rnd=183 T=25.5 H=54.5 t=34s PWM=4095 ext=0% | OK
```

WiFi, sensor, perilla y envíos: **bien**. Touch: **el LED externo no se movía de 0 %**.

Problema 5 (el más importante): el touch no detectaba el dedo

Cómo lo investigamos: cargamos un programa de diagnóstico que sólo imprime los números crudos de `touchRead()`, cuatro veces por segundo, mientras se tocaban los pines.

Pin	Sin tocar	Tocando	Cuánto baja
D13 (GPIO13)	1129 – 1132	785 – 1000 aprox.	10 % a 30 %
D4 (GPIO4)	937 – 940	hasta 796	hasta 15 %
D15, D12, D14, D27 (control)	972 / 1200 / 1286 / 1450	casi sin cambios	≈ 1 % (no se tocaron)

La causa: el código original consideraba "tocado" sólo cuando la lectura bajaba **por debajo del 60 %** del valor en reposo (o sea, una baja del 40 %). En esta placa el dedo baja la lectura sólo un 10–30 %, así que **nunca llegaba**.

La solución: cambiar el umbral al **90 %** (basta una baja de más del 10 %). Sin tocar, la lectura varía menos del 1 %, así que no se activa sola.

```
// ANTES                                  // DESPUÉS
return (suma / 20) * 60 / 100;            return (suma / 20) * 90 / 100;
```

Con el cambio, los umbrales calculados al arrancar fueron `+1017 -846` (90 % de ~1130 y ~940), y el LED externo pasó de **0 % → 100 % → 0 %** tocando D13 y después D4. **Resuelto.**

Nota: en el primer arranque en la placa real, los umbrales calculados habían sido mucho más bajos (+194 / −489). Probablemente la placa se estaba manipulando justo durante la calibración. Moraleja: no tocar nada los primeros segundos.

Problema 6: algunos envíos con error -301 o -401

Código	Qué significa	Por qué pasó
`-301`	No se pudo conectar con ThingSpeak	Cortes momentáneos de la red de la facu. Desde la compu ThingSpeak respondía bien (HTTP 200 en 0,6–1,1 s).
`-401`	ThingSpeak no guardó el dato porque llegó "demasiado rápido" (menos de 15 s desde el anterior)	Con envíos cada 16 s hay sólo 1 s de margen. Si la red demora un envío, el siguiente llega antes de tiempo.

No son errores del código: el programa sigue y el siguiente envío sale bien. No se puede agrandar el intervalo porque la consigna pide 16 s.

Etapa 8 · Sketch 2 por WiFi (OTA)

1. Comprobamos que la compu (`172.16.18.127`) y la placa (`172.16.18.63`) estuvieran en la **misma red** y se "vieran": ping OK (45–237 ms).
2. Abrimos el firewall de Windows para la herramienta de carga por red (`espota.exe`), regla "ESP32 OTA (espota)".
3. El primer intento falló porque la consola pedía una contraseña de OTA ("user input not supported in non interactive mode"). El programa no tiene contraseña, así que le pasamos una vacía.
4. Subida: `Uploading: [=====] 100% Done...` ✓ — en la pantalla se vio "Actualizando OTA...".

Etapa 9 · Sketch 2 funcionando

```
Lectura OK: 349 | 26.5 | 46.9 | 354 | 0 | 0
```

	F1	F2	F3	F4	F5	F6
Leído por la placa	349	26.5	46.9	354	0	0
Guardado en ThingSpeak (registro 38)	349	26.5	46.9	354	0	0

Coincide exactamente ✓. Se repetía en cada lectura porque, con el Sketch 2 cargado, ya nadie mandaba datos nuevos (es lo esperable). Pantallas rotando ✓ y **31 pulsaciones** contadas al apretar el eje plateado ✓.

Etapa 10 · Dejar lista la entrega

- WiFi **CIDS** en los dos sketches (decisión del grupo: se evalúa en la facu).
- Integrantes y legajos agregados al encabezado de los dos `.ino` .
- Guion y presentación corregidos: umbral 90 % y fechas de evaluación posibles.
- Compilación final de los dos sketches: sin errores.

8. Todas las mediciones

Compilación

Sketch	Memoria de programa	Memoria dinámica
Envío	1.026.390 bytes (78 %)	54.184 bytes (16 %)
Lectura	1.002.891 bytes (76 %)	54.120 bytes (16 %)

Envíos del Sketch 1 en la placa real

Prueba	t (s)	Aleatorio	T (°C)	H (%)	PWM int	LED ext	Resultado
1ª (umbral 60 %)	18	130	25.5	55.1	2833	0 %	OK
	34	183	25.5	54.5	4095	0 %	OK
2ª (umbral 90 %)	18	156	27.1	50.6	0	0 %	OK
	34	443	27.2	49.1	0	0 %	-401
	50	386	27.1	48.8	0	0 %	OK
3ª (tocando)	34	342	26.8	49.5	0	0 %	OK
	50	279	26.8	49.6	0	0 %	-301
	66	282	26.7	52.5	0	**100 %**	-301
4ª (D13 y D4)	130	156	26.9	49.0	0	0 %	OK
	146	440	26.9	48.9	0	**100 %**	-401
	162	331	27.0	48.2	0	**0 %**	OK

PWM = 0 en varias pruebas porque la perilla quedó al mínimo; al girarla, el valor en pantalla cambiaba (confirmado mirando la placa).

Lecturas crudas de los pines touch (diagnóstico)

Pin	Reposo	Mínimo tocando	Baja máxima	Umbral con 60 %	Umbral con 90 %
D13	≈ 1130	785	≈ 30 %	678 (inalcanzable)	1017 ✓
D4	≈ 940	796	≈ 15 %	564 (inalcanzable)	846 ✓

Red y ThingSpeak

Medición	Valor
IP de la placa (CIDS)	172.16.18.63

IP de la notebook (CIDS)	172.16.18.127 (/24, misma red)
Ping compu → placa	45 / 237 / 101 ms
Respuesta de ThingSpeak desde la compu	HTTP 200 en 1,11 / 0,74 / 0,65 s
Bandas de CIDS	5 GHz (canales 149 y 157) y 2.4 GHz (canales 1 y 6)
Registros en el canal al final de las pruebas	38

Simulación (Wokwi)

Prueba	Resultado
Envíos	3 de 3 OK (IP virtual 10.13.37.2)
Perilla a 0.75	PWM = 3071
Sensor a 30.2 °C / 68 %	T=30.2, H=68.0 enviados
Lectura	402 | 30.2 | 68.0 | 24 | 3071 | 0 (coincide)
Pulsador	3 clics → "Pulsaciones: 3"

9. Problemas y soluciones (resumen)

#	Síntoma	Causa	Solución
1	Había un Arduino 1.8.19 abierto	Versión vieja de la Microsoft Store	Instalar y usar Arduino IDE 2.3.10
2	La placa aparece con error y sin puerto COM (código 28)	Falta el driver del chip CP2102; la notebook es ARM64	Driver Silicon Labs 11.5.0.417 del Catálogo de Microsoft Update, instalado como administrador → COM11
3	La descarga del driver da "Access Denied"	La web del fabricante bloquea descargas automáticas	Usar el Catálogo de Microsoft (o bajarlo con el navegador)
4	"Wrong boot mode (0x13)" / se queda en Connecting...	La placa no entró en modo carga	BOOT apretado + apretar y soltar EN + soltar BOOT
5	"No such file or directory" al compilar	La carpeta se movió a `TP-de-TPA`	Usar la ruta nueva
6	LED azul prendido en la ESP32 suelta	Pin de la perilla sin nada conectado (lee ruido)	Ninguna: es normal sin la placa de la cátedra
7	El touch no mueve el LED externo	Umbral de 60 % demasiado exigente para esta placa	Umbral al 90 % (medido con diagnóstico)
8	Envíos con -301	Cortes de la red de la facu	Ninguna: el siguiente envío sale bien
9	Envíos con -401	Llegó antes de los 15 s que exige ThingSpeak (por demora de red)	Ninguna: la consigna pide 16 s; es ocasional
10	OTA pide contraseña en la consola	La herramienta pregunta siempre	Pasar contraseña vacía (en el IDE no pasa: sólo se elige el puerto de red)
11	Simulación colgada	Foto pedida después del fin del escenario	Alargar el escenario

10. Qué hacer el día de la evaluación

Antes

- Mirar en **Autogestión** qué día les toca (puede ser el 29/10).
- Llevar **esta notebook** (ya tiene IDE, placa ESP32, librerías, driver y permiso de firewall).
- Llevar un **cable micro USB de datos** y un **celular con hotspot en 2.4 GHz** como plan B.
- Estar los **6 integrantes**.

Paso a paso

1. Conectar la **notebook al WiFi CIDS** y la placa por USB. Abrir Arduino IDE 2.
2. *Herramientas* → *Placa* → *esp32* → **DOIT ESP32 DEVKIT V1**. *Herramientas* → *Puerto* → el **COM** que aparezca.
3. Abrir `sketch_envio_grupo_24.ino` → **Subir**. Si se queda en "Connecting...": BOOT + EN, soltar BOOT.
4. **No tocar D13 ni D4** los primeros segundos. Abrir el Monitor Serie a 115200 si quieren ver los envíos.
5. Mostrar: perilla negra → LED azul y "PWM LED int"; D13 / D4 → LED externo y "%"; temperatura y humedad; "[OK]" cada 16 s; los gráficos de ThingSpeak sumando puntos.
6. *Herramientas* → *Puerto* → elegir el puerto de red **esp32-grupo24**. Abrir `sketch_lectura_grupo_24.ino` → **Subir**. Si Windows pregunta por el firewall: Permitir. En la pantalla: "Actualizando OTA...".
7. Mostrar: pantallas 1/2 y 2/2 que rotan cada 4 s, valores leídos del canal, "Pulsaciones" subiendo al apretar el eje plateado, "Leido hace Xs".

> **Si no hay WiFi CIDS ese día**
>
> Cambiar `WIFI_SSID` y `WIFI_PASS` en **los dos** sketches por los del hotspot (2.4 GHz), conectar la notebook a ese mismo hotspot, y empezar de nuevo desde el paso 3. Si no aparece el puerto `esp32-grupo24`, casi siempre es porque la compu y la placa no están en la misma red.

11. Preguntas probables

Pregunta	Respuesta corta
¿Por qué cada 16 segundos?	ThingSpeak gratis acepta como máximo una escritura cada 15 s. 16 deja 1 s de margen.
¿Por qué los 6 campos juntos?	Con 6 escrituras separadas ThingSpeak rechazaría casi todas por el límite de 15 s. `setField` carga, `writeFields` manda todo junto.
¿Qué es PWM de 12 bits?	Regular el brillo prendiendo y apagando muy rápido, con 4096 niveles (0 a 4095).
¿Por qué el valor de la perilla va directo al LED?	El ADC lee en 12 bits (0–4095) y el PWM también es de 12 bits: mismo rango, no hay que convertir.
¿Cómo funciona el touch?	El pin mide capacitancia; al tocarlo con el dedo el valor **baja**. Al arrancar se mide el reposo y se fija un umbral.
¿Por qué el umbral es 90 %?	Lo medimos: en esta placa el dedo baja la lectura sólo 10–30 %. Con 60 % nunca se detectaba. En reposo varía < 1 %, así que 90 % no da falsos toques.
¿Qué es OTA?	Cargar el programa por WiFi. El Sketch 1 abre un puerto de red; el IDE lo muestra y se sube como si fuera USB.
¿Por qué no usan delay()?	Bloquea el loop: no se atendería el OTA ni se leerían los sensores mientras tanto. Usamos `millis()`.
¿Por qué interrupción en el pulsador?	Leer ThingSpeak tarda ~1 s; con interrupción no se pierden pulsaciones durante ese tiempo.
¿Qué es el antirrebote?	Ignorar pulsos a menos de 200 ms del anterior, porque el contacto vibra al apretar.
¿Por qué el Sketch 2 no necesita clave?	El canal es público: cualquiera puede leerlo. La Write API Key sólo hace falta para escribir.
¿Qué significan -301 y -401?	-301: no pudo conectar (red). -401: ThingSpeak no guardó el punto por llegar antes de 15 s. Ambos ocasionales en una red lenta.
¿Por qué la ESP32 no se conecta a una red de 5 GHz?	Su radio WiFi sólo funciona en 2.4 GHz.

12. Pendientes y advertencias

- **El video** (`guia/TPI_Grupo24_explicacion.mp4`) todavía dice "umbral del 60 %" y "exponemos el jueves 19". El guion (`guia/guion.txt`) y la presentación ya están corregidos, por si lo regraban.
- **La ESP32 suelta** quedó con el Sketch 1 y el WiFi CIDS. Si se enciende cerca, va a escribir en el mismo canal y provocar errores -401. No la prendan.
- **La placa de la cátedra** quedó con el Sketch 2 cargado. Para la evaluación se arranca de nuevo desde el Sketch 1 por USB.
- **La clave del WiFi CIDS** queda escrita en los archivos que se entregan (inevitable, porque se evalúa lo subido).
- **El token de Wokwi** se compartió durante el trabajo. Si les preocupa, generen uno nuevo en wokwi.com/dashboard/ci.
- El **PASO A PASO** original no incluye lo aprendido (driver ARM64, BOOT + EN, errores de red). Esta bitácora lo cubre.

13. Anexo: archivos del proyecto y comandos usados

Carpeta del proyecto

`C:\Users\gonza\OneDrive\Desktop\TPI_Grupo24\TPI_Grupo24\TP-de-TPA\`

Archivo / carpeta	Qué es	¿Se entrega?
`sketch_envio_grupo_24\sketch_envio_grupo_24.ino`	Sketch 1	**Sí**
`sketch_lectura_grupo_24\sketch_lectura_grupo_24.ino`	Sketch 2	**Sí**
`integrantes_grupo_24.txt`	Integrantes, legajos, Channel ID, Write API Key	**Sí**
`TPI 4K3.pdf`	La consigna	No
`PASO A PASO - probar con la placa.md`	Instructivo original	No
`guia\`	Presentación, guion y video para estudiar	No
`simulacion_wokwi\`	Placas virtuales, script de preparación, escenarios de prueba y capturas	No
`Bitacora_TPI_Grupo24.pdf`	Este documento	No

Dónde quedó instalado cada cosa

Qué	Dónde
Arduino IDE 2.3.10	`%LOCALAPPDATA%\Microsoft\WinGet\Packages\ArduinoSA.IDE.stable_...\Arduino IDE.exe`
arduino-cli 1.5.1	Dentro del IDE: `resources\app\lib\backend\resources\arduino-cli.exe`
Paquete esp32 3.3.12	`%LOCALAPPDATA%\Arduino15\packages\esp32`
Librerías	Carpeta de librerías de Arduino del usuario
wokwi-cli 0.29.0	`%USERPROFILE%\.wokwi\bin\wokwi-cli.exe`
Extensión Wokwi para VS Code	`wokwi.wokwi-vscode` 4.0.1
Driver CP210x	Instalado en Windows (oem25.inf)
Regla de firewall	"ESP32 OTA (espota)" — entrada permitida para espota.exe

Comandos principales (equivalen a los botones del IDE)

```
# Instalar el IDE
winget install --id ArduinoSA.IDE.stable --exact

# Soporte ESP32 y librerías
arduino-cli config add board_manager.additional_urls https://espressif.github.io/arduino-
esp32/package_esp32_index.json
arduino-cli core install esp32:esp32
arduino-cli lib install "DHT sensor library" "Adafruit Unified Sensor" "Adafruit SH110X" "Adafruit
GFX Library" "ThingSpeak"

# Compilar y subir por USB (botón Subir con un puerto COM)
arduino-cli compile --fqbn esp32:esp32:esp32doit-devkit-v1 --upload -p COM11 sketch_envio_grupo_24

# Subir por WiFi / OTA (botón Subir con el puerto de red)
arduino-cli upload --fqbn esp32:esp32:esp32doit-devkit-v1 -p 172.16.18.63 --protocol network --
upload-field password= sketch_lectura_grupo_24

# Instalar el driver (como administrador)
pnputil /add-driver silabser.inf /install

# Simulación
powershell -ExecutionPolicy Bypass -File simulacion_wokwi\preparar_simulacion.ps1
wokwi-cli --scenario envio\prueba_envio.yaml simulacion_wokwi\envio
```